

DevMatch: Especificação de Requisitos e Modelagem de Projeto de uma Plataforma de Contratação de Profissionais de TI

DevMatch: Requirements Specification and Software Design Modeling for an IT Professionals Marketplace

Bernardo Augusto – Pontifícia Universidade Católica de Minas Gerais – PUC Minas
Cleverson Resende – Pontifícia Universidade Católica de Minas Gerais – PUC Minas
Rafael Lopes – Pontifícia Universidade Católica de Minas Gerais – PUC Minas
Matheus Augusto – Pontifícia Universidade Católica de Minas Gerais – PUC Minas
Kaio Ferreira – Pontifícia Universidade Católica de Minas Gerais – PUC Minas

Curso de Engenharia de Software – PUC Minas. Disciplina: Projeto de Software. Professora: Milena Menezes Adão.

1 Introdução

O desenvolvimento de software sob demanda envolve a contratação de profissionais com competências técnicas específicas, a negociação de escopo, prazo e orçamento, o acompanhamento das entregas e a necessidade de segurança no fluxo de pagamentos. Nesse contexto, mecanismos claros de acompanhamento, registro e proteção financeira são relevantes para reduzir desacordos entre quem contrata o serviço e quem o executa.

O sistema-base deste trabalho é o **DevMatch**, uma plataforma de *marketplace* que conecta profissionais de TI freelancers a clientes – empresas ou pessoas físicas – que necessitam contratar serviços de desenvolvimento de software. A plataforma organiza o processo de contratação desde a publicação de projetos e o envio de propostas até a formalização do contrato, o acompanhamento de marcos de entrega (*milestones*) e a liberação de pagamentos.

Entre as funcionalidades centrais do DevMatch estão a gestão de projetos e propostas, a negociação entre Cliente e Freelancer, a formalização e o acompanhamento de contratos, o pagamento em custódia (*escrow*), a abertura de disputas e a avaliação bidirecional ao final do contrato. O sistema também prevê *matching* inteligente, que recomenda projetos e profissionais conforme critérios de compatibilidade, e verificação de competências por meio de testes técnicos automatizados.

O objetivo deste trabalho é especificar e modelar o DevMatch sob a perspectiva da Engenharia de Software, mantendo consistência entre requisitos e decisões de projeto. Conforme o cronograma da disciplina, esta entrega parcial apresenta a descrição dos atores, o modelo de casos de uso e as histórias de usuário, a arquitetura do sistema, os diagramas de componentes e de implantação e o diagrama de classes. Os diagramas de sequência do sistema, contratos de operação, diagramas comportamentais adicionais e a modelagem de dados serão desenvolvidos nas entregas subsequentes.

2 Modelagem de Requisitos

2.1 Descrição de Atores

Os atores foram identificados a partir dos papéis e responsabilidades definidos para o DevMatch. A Tabela 1 sintetiza os quatro perfis humanos que interagem diretamente com a plataforma.

Cliente e Freelancer participam diretamente do ciclo de contratação. Administrador do Sistema e Analista Financeiro/Suporte exercem funções internas de governança, mediação, auditoria e operação financeira. A separação desses papéis orienta as permissões de acesso e reduz sobreposição de responsabilidades.

2.2 Modelo de Casos de Uso e Histórias de Usuário

O modelo de casos de uso representa as principais interações dos atores com o DevMatch. Os casos foram derivados das funcionalidades de gestão de projetos, contratação e acompanhamento, pagamento e segurança financeira, avaliação e reputação, *matching* e verificação de competências.

Para preservar a legibilidade, o modelo foi dividido em três visões complementares: Freelancer, Cliente e atores internos. Essa divisão não cria funcionalidades duplicadas; casos compartilhados, como acompanhar contrato, abrir disputa e avaliar a contraparte, representam o mesmo comportamento visto por diferentes atores.

As Figuras 1–3 mantêm os mesmos identificadores e nomes da Tabela 2. A aprovação de um marco é responsabilidade do Cliente; a gestão da custódia e dos repasses é responsabilidade do Analista Financeiro/Suporte; e a mediação de uma disputa é executada pelo Administrador do Sistema, embora a abertura possa ser iniciada por Cliente ou Freelancer.

As histórias de usuário complementam os casos de uso ao explicitar o valor esperado por cada papel.

3 Modelagem de Projeto

3.1 Arquitetura do Sistema

Para o DevMatch, propõe-se uma arquitetura em camadas com módulos de negócio bem delimitados. A

Table 1. Atores do sistema DevMatch.

Ator	Objetivo no sistema	Principais responsabilidades e interações
Freelancer	Encontrar oportunidades e executar serviços de desenvolvimento de software.	Criar e editar perfil profissional; informar <i>stack</i> , portfólio, certificações e valor/hora; buscar projetos; enviar e negociar propostas; acompanhar contratos e marcos; realizar testes técnicos; receber pagamentos liberados e avaliar clientes.
Cliente	Contratar profissionais para executar projetos de software.	Publicar projetos com escopo, prazo e orçamento; analisar e aceitar propostas; formalizar contratos; acompanhar e aprovar marcos; autorizar liberações de pagamento; abrir disputas e avaliar freelancers.
Administrador do Sistema	Garantir governança, segurança, integridade e rastreabilidade da plataforma.	Gerenciar usuários e permissões; apoiar a verificação de identidade; mediar disputas; monitorar logs, auditorias de acesso e transações relevantes.
Analista Financeiro/Suporte	Operar processos financeiros e prestar suporte especializado.	Gerenciar custódia e repasses; emitir relatórios financeiros e de comissões; atender chamados relacionados a pagamentos.

Table 2. Casos de uso identificados para o DevMatch.

ID	Caso de uso	Ator principal / atores principais
UC01	Autenticar-se na plataforma	Todos os atores humanos
UC02	Criar e editar perfil profissional	Freelancer
UC03	Buscar projetos publicados	Freelancer
UC04	Enviar e negociar proposta	Freelancer
UC05	Realizar teste de competência	Freelancer
UC06	Publicar projeto	Cliente
UC07	Analisar e aceitar propostas	Cliente
UC08	Formalizar contrato e definir marcos	Cliente, Freelancer
UC09	Acompanhar contrato e marcos de entrega	Cliente, Freelancer
UC10	Aprovar marco e autorizar liberação de pagamento	Cliente
UC11	Comunicar-se durante o projeto	Cliente, Freelancer
UC12	Consultar recomendações de <i>matching</i>	Cliente, Freelancer
UC13	Abrir disputa	Cliente, Freelancer
UC14	Avaliar a contraparte ao final do contrato	Cliente, Freelancer
UC15	Gerenciar custódia e repasses	Analista Financeiro/Suporte
UC16	Emitir relatórios financeiros e de comissões	Analista Financeiro/Suporte
UC17	Atender chamado relacionado a pagamento	Analista Financeiro/Suporte
UC18	Gerenciar usuários e permissões	Administrador do Sistema
UC19	Mediar disputa	Administrador do Sistema
UC20	Monitorar logs e auditorias	Administrador do Sistema

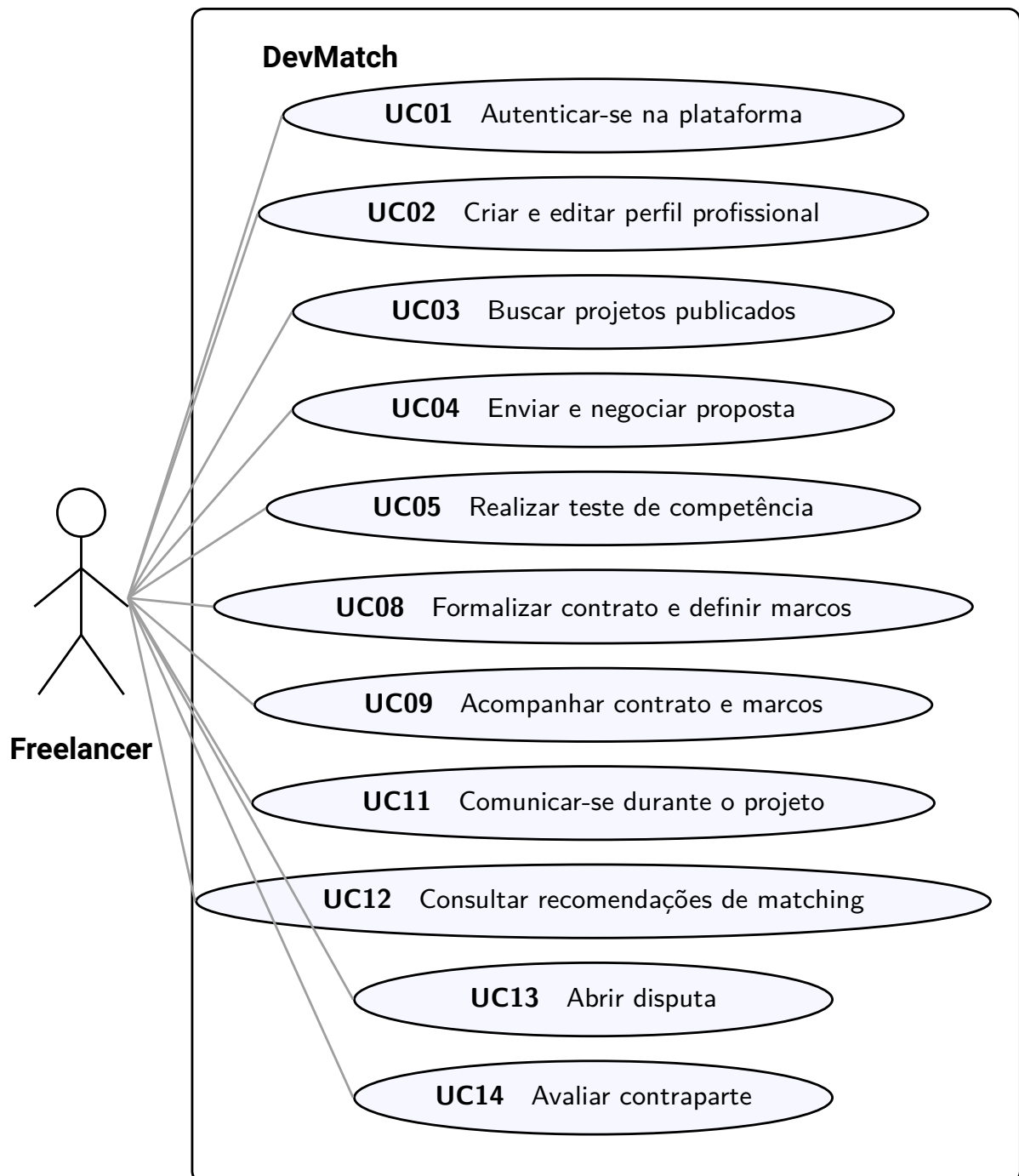


Figure 1. Casos de uso associados ao ator Freelancer.

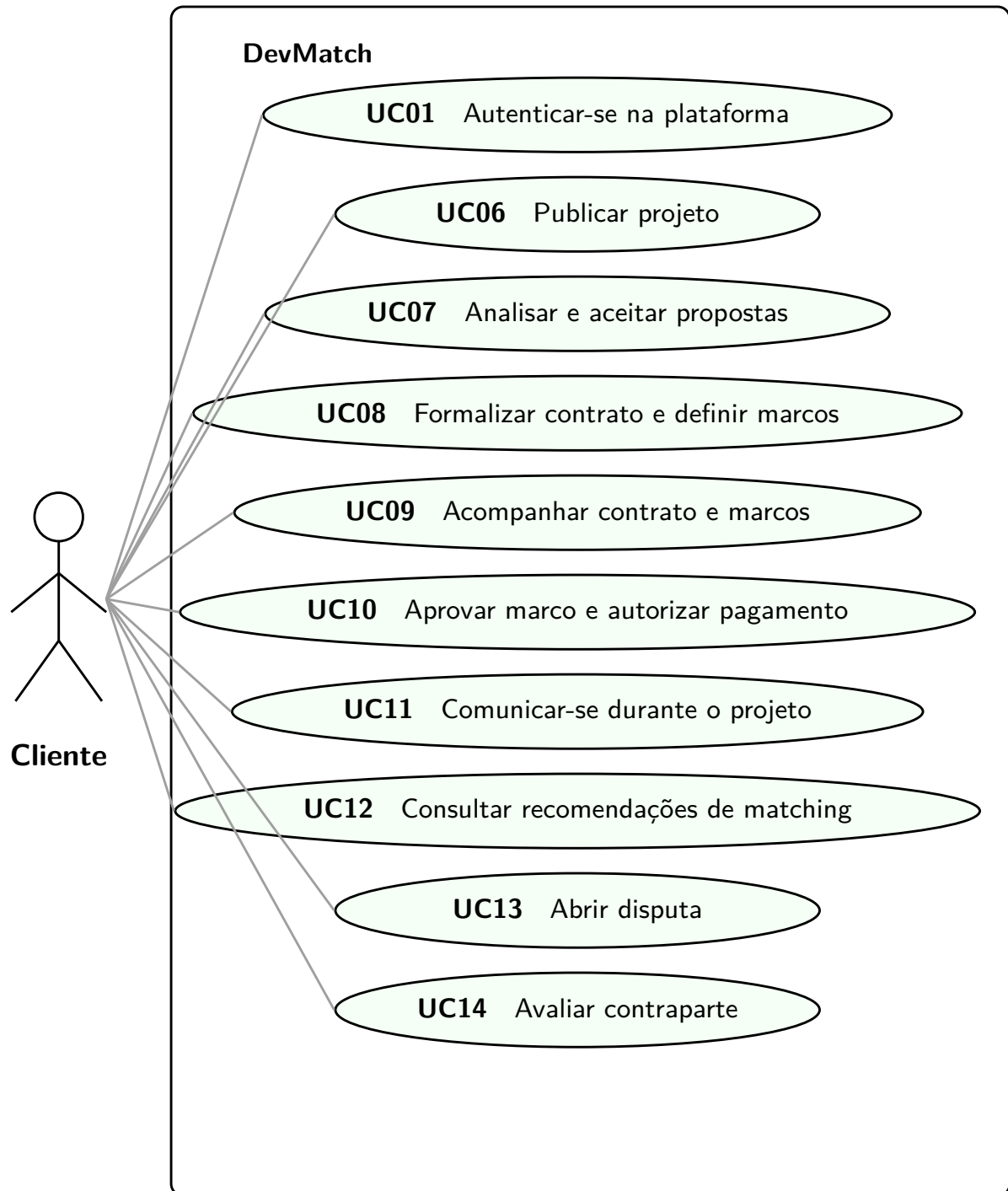


Figure 2. Casos de uso associados ao ator Cliente.

Table 3. Histórias de usuário do DevMatch.

UC	História de usuário
UC02	Como Freelancer, quero cadastrar e atualizar meu perfil profissional, para que os clientes conheçam minhas competências, portfólio, certificações e valor/hora.
UC03	Como Freelancer, quero buscar projetos publicados, para que eu encontre oportunidades compatíveis com minhas competências.
UC04	Como Freelancer, quero enviar e negociar uma proposta de prazo e valor, para que eu possa concorrer à contratação de um projeto.
UC05	Como Freelancer, quero realizar testes técnicos, para que minhas competências possam ser verificadas e representadas por selos no perfil.
UC06	Como Cliente, quero publicar um projeto com escopo, prazo, orçamento e <i>stack</i> desejada, para que freelancers qualificados possam enviar propostas.
UC07	Como Cliente, quero analisar e aceitar propostas recebidas, para que eu escolha o profissional mais adequado ao projeto.
UC08	Como Cliente ou Freelancer, quero formalizar um contrato com marcos de entrega definidos, para que o trabalho possua escopo e critérios de acompanhamento claros.
UC09	Como Cliente ou Freelancer, quero acompanhar os marcos do contrato, para que eu saiba o andamento e as próximas entregas do projeto.
UC10	Como Cliente, quero aprovar um marco concluído, para que o pagamento correspondente possa ser liberado ao Freelancer.
UC11	Como Cliente ou Freelancer, quero manter a comunicação vinculada ao projeto, para que decisões e alinhamentos fiquem registrados.
UC12	Como Cliente, quero receber recomendações de freelancers compatíveis com meu projeto, para reduzir o esforço de busca por profissionais adequados.
UC12	Como Freelancer, quero receber recomendações de projetos compatíveis com meu perfil, para encontrar oportunidades mais relevantes.
UC13	Como Cliente ou Freelancer, quero abrir uma disputa em caso de desacordo, para que o problema possa ser analisado pela plataforma.
UC14	Como Cliente ou Freelancer, quero avaliar a contraparte ao final do contrato, para que o histórico de reputação reflita experiências anteriores.
UC15	Como Analista Financeiro/Suporte, quero gerenciar a custódia e os repasses, para que os valores sejam movimentados conforme a aprovação dos marcos.
UC16	Como Analista Financeiro/Suporte, quero emitir relatórios financeiros e de comissões, para que as operações possam ser acompanhadas e auditadas.
UC17	Como Analista Financeiro/Suporte, quero atender chamados relacionados a pagamentos, para que problemas financeiros sejam tratados pela equipe responsável.
UC18	Como Administrador do Sistema, quero gerenciar usuários e permissões, para que cada perfil tenha acesso somente às funções autorizadas.
UC19	Como Administrador do Sistema, quero mediar disputas, para que conflitos sejam tratados de forma registrada e imparcial.
UC20	Como Administrador do Sistema, quero monitorar logs e auditorias, para que ações relevantes e transações possam ser rastreadas.

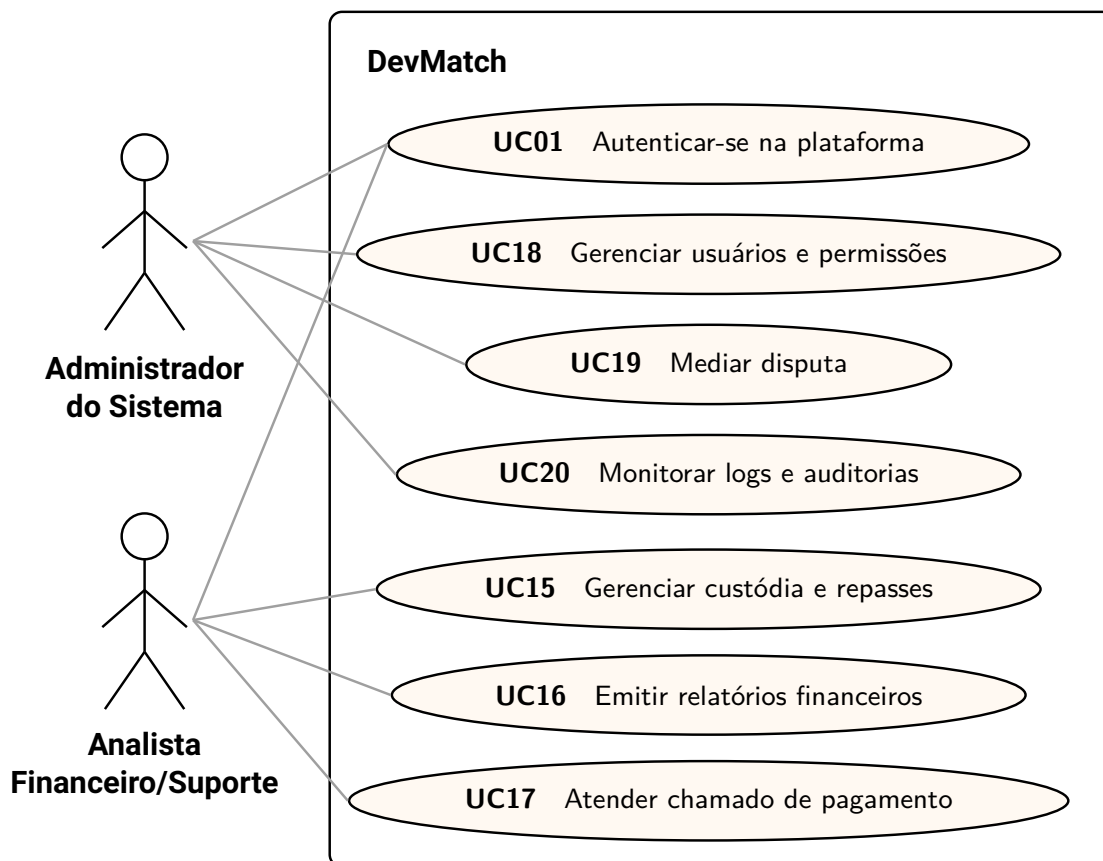


Figure 3. Casos de uso associados aos atores internos Administrador do Sistema e Analista Financeiro/Suporte.

organização separa apresentação, controle de acesso, regras de negócio, persistência/auditoria e integrações externas. Essa divisão reduz o acoplamento entre responsabilidades e permite que módulos críticos, como contratos e pagamentos, evoluam de forma independente.

A arquitetura é organizada nas seguintes responsabilidades: (i) **Apresentação**, composta pelas interfaces Web e móvel; (ii) **API e Controle de Acesso**, responsável por autenticação, autorização e roteamento; (iii) **Serviços de Negócio**, com módulos de Usuários, Projetos/Propostas, Contratos/Marcos, Pagamentos/Escrow, Matching, Avaliação/Reputação, Administração e Suporte; (iv) **Persistência e Auditoria**, responsável pelo armazenamento dos dados e registros rastreáveis; e (v) **Integrações Externas**, utilizadas para pagamento, notificações e verificação de identidade.

A Figura 4 evidencia a separação entre as responsabilidades e o fluxo principal entre as camadas. A modularização favorece escalabilidade e disponibilidade por permitir replicação da aplicação, reforça segurança e privacidade ao centralizar controles de acesso e sustenta a auditabilidade por manter registros das operações críticas.

3.2 Diagramas de Componentes e de Implantação

3.2.1 Diagrama de Componentes

O diagrama de componentes refina a arquitetura ao representar os principais módulos de software e suas dependências. A interface Web/Móvel consome a API da plataforma, que encaminha as requisições aos componentes de domínio. Os componentes de negócio acessam persistência e auditoria e utilizam adaptadores específicos quando precisam se comunicar com provedores externos.

Na Figura 5, Projetos/Propostas, Contratos/Marcos, Pagamentos/Escrow e Avaliação/Reputação permanecem separados conceitualmente. Essa divisão mantém correspondência com os grupos funcionais dos requisitos e evita concentrar responsabilidades distintas em um único componente.

3.2.2 Diagrama de Implantação

O diagrama de implantação descreve a distribuição dos artefatos em nós de execução. A proposta considera clientes Web e móvel acessando a plataforma por HTTPS, um balanceador de carga, uma camada de API, múltiplas instâncias da aplicação, persistência com réplica, *cache*, barramento de eventos e serviços externos fora do ambiente controlado pelo DevMatch.

A Figura 6 explicita os nós de execução e os principais canais de comunicação. As múltiplas instâncias da aplicação e a réplica do banco de dados são coerentes

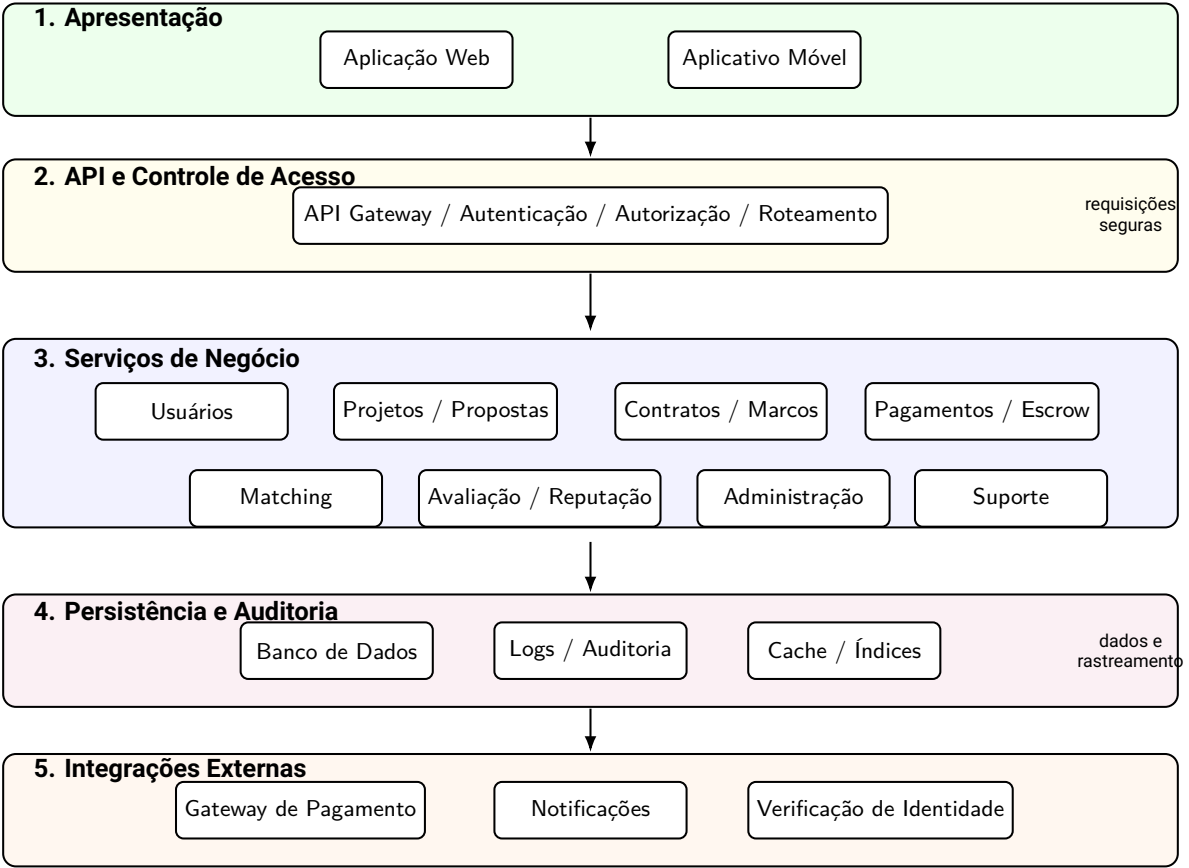


Figure 4. Arquitetura proposta para o sistema DevMatch.

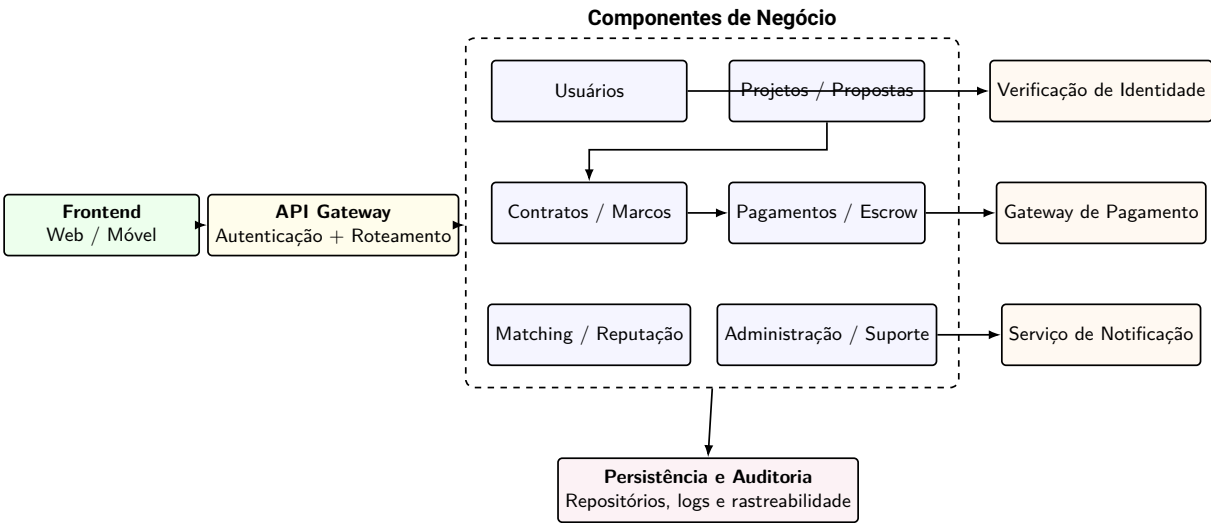


Figure 5. Diagrama de componentes do DevMatch.

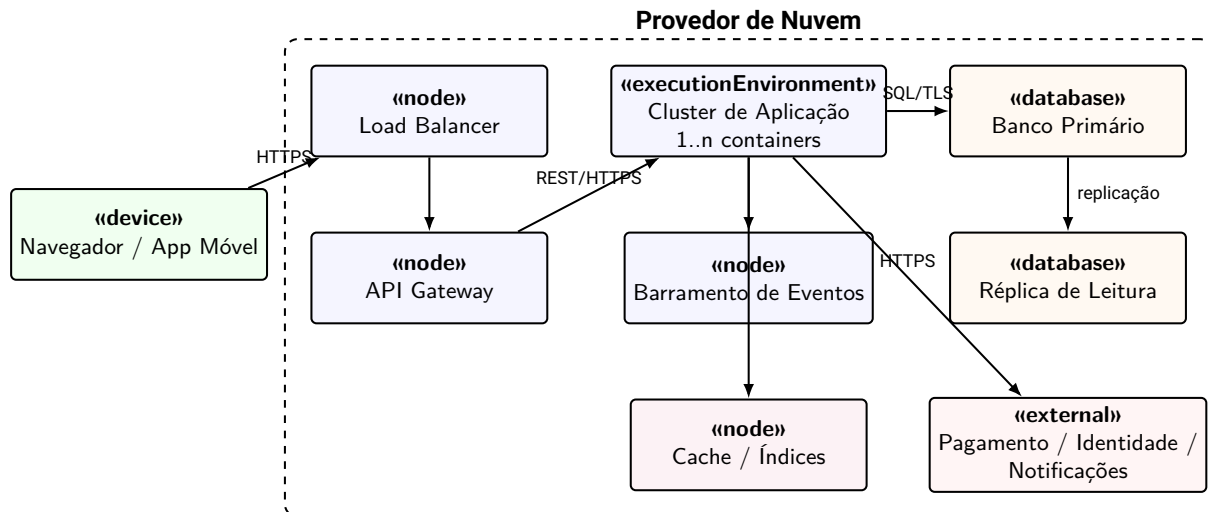


Figure 6. Diagrama de implantação do DevMatch.

com os requisitos de disponibilidade, escalabilidade e tolerância a falhas previstos para o sistema.

3.3 Diagrama de Classes

O diagrama de classes apresenta uma visão estrutural resumida do domínio do DevMatch. A classe abstrata **Usuario** concentra dados comuns e é especializada em **Freelancer**, **Cliente**, **Administrador** e **AnalistaFinanceiro**. O Cliente publica **Projeto**; cada Projeto recebe **Proposta** enviadas por Freelancers; uma proposta aceita pode originar um **Contrato**; e cada Contrato contém um ou mais **Marco** de entrega.

O fluxo financeiro é representado por **ContaEscrow** associada ao Contrato e por **Pagamento** relacionado aos marcos aprovados. O Contrato também pode possuir **Avaliacao** e **Disputa**. A Disputa pode ser mediada por um Administrador, enquanto o AnalistaFinanceiro atua na gestão da custódia e dos repasses.

As multiplicidades apresentadas no diagrama indicam que um Cliente pode publicar vários Projetos; um Projeto pode receber várias Propostas; cada Proposta é enviada por um Freelancer; um Contrato relaciona Cliente e Freelancer no contexto de uma proposta aceita; e um Contrato contém um ou mais Marcos. O modelo central foi mantido enxuto para preservar a legibilidade no corpo do artigo.

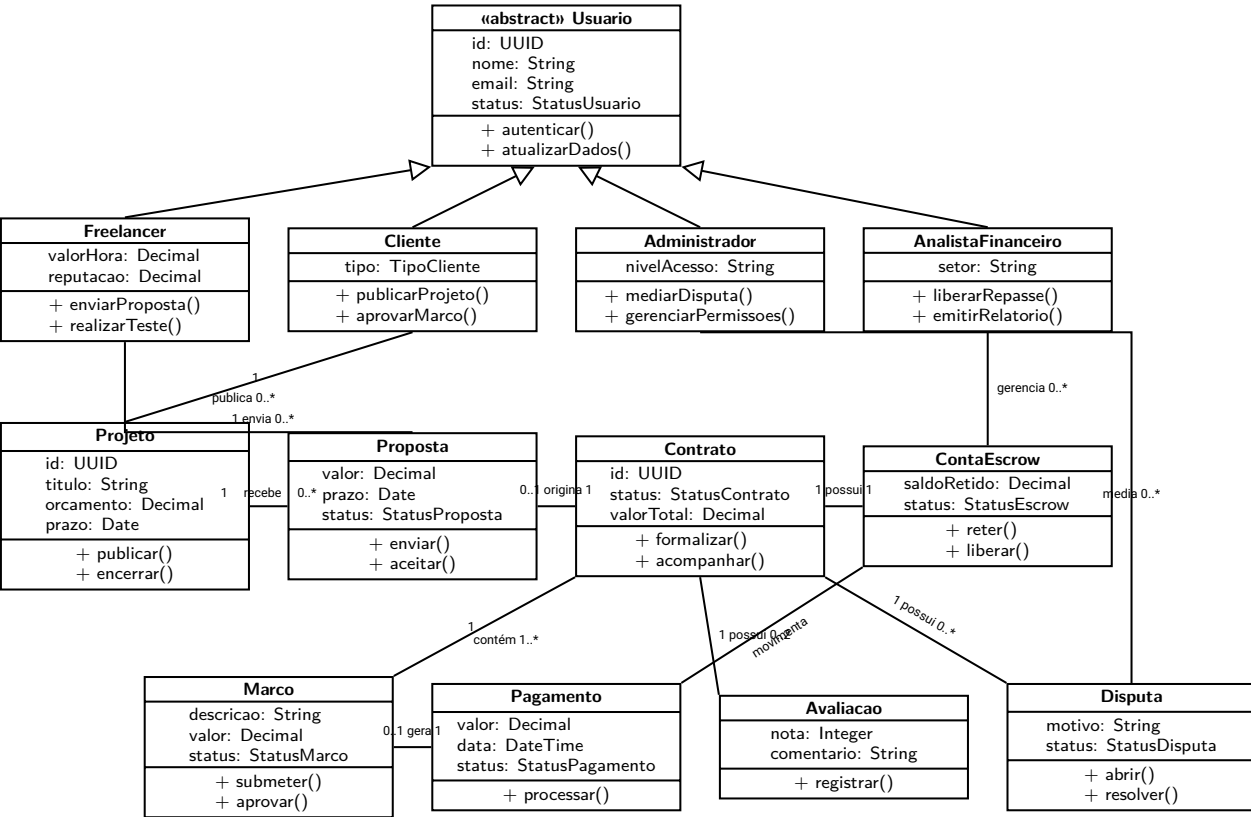


Figure 7. Diagrama de classes resumido do domínio do DevMatch.